

Laboratorio IA 7: Red neuronal pequena y evolucion por algoritmo genetico

Materia: Inteligencia Artificial

Dinamica: practica en terminal **Entregable:** experimento, codigo, presentacion de resultados y conclusiones

Construir una red neuronal pequena con PyTorch que controle al robot de percepcion local y evolucionar sus pesos mediante un algoritmo genetico. El mejor individuo se guardara en disco para probarlo posteriormente sobre un mapa modificado a mano.

1. Continuidad con las clases anteriores

El laboratorio conserva el mundo del robot usado en las sesiones anteriores: una cuadrícula, un punto de inicio, una meta, obstaculos, sensores locales y una funcion de fitness. La progresion es:

- En las primeras clases se evoluciono una secuencia de movimientos.
- Despues se incorporaron obstaculos y se compararon mutacion y cruce.
- Con la percepcion local, el robot paso a consultar el entorno antes de decidir.
- En este laboratorio se conserva esa politica reactiva, pero se reemplaza la tabla de reglas por una red neuronal.

El algoritmo evolutivo sigue seleccionando comportamientos por fitness. Lo que cambia es la representacion del individuo: antes eran comandos o reglas; ahora son los pesos de una red neuronal.

2. Preparacion del entorno

El programa necesita Python, PyTorch y el archivo `robot_red_neuronal_evolutiva.py`. Con el interprete del laboratorio se puede comprobar la instalacion:

```
python3 -c "import torch; print(torch.__version__)"
```

Si PyTorch no esta instalado en ese interprete:

```
python3 -m pip install --index-url https://download.pytorch.org/whl/cpu torch
```

3. Mapa y sensores

El mapa se modifica directamente al principio del archivo Python:

```
TAMANO = 8
INICIO = (0, 0)
META = (7, 7)
OBSTACULOS = {
    (0, 3), (1, 3), (2, 0), (2, 2), (2, 3),
    (4, 5), (5, 2), (5, 3), (5, 4), (6, 6),
}
```

En cada paso el robot consulta las celdas vecinas en orden U D L R. Una celda libre se representa con 0; un borde o un obstaculo, con 1. Por tanto, la entrada de la red tiene cuatro valores binarios. El robot no observa el mapa completo ni recibe directamente la posicion de la meta.

4. La red neuronal

La arquitectura es deliberadamente pequena:

Entrada	Capa oculta	Salida
4 sensores U,D,L,R	8 neuronas con ReLU combinacion no lineal	4 acciones un valor por accion

La salida no es una accion directamente. Son cuatro valores llamados logits. La accion elegida es la que tenga el valor maximo:

```
logits = red(entrada)[0]
indice = int(torch.argmax(logits).item())
accion = COMANDOS[indice]
```

Cada individuo de la poblacion es una instancia distinta de `RedPercepcion`. Sus pesos funcionan como el material genetico.

5. Fitness y algoritmo evolutivo

Cada red recorre el mapa durante como maximo 30 pasos. El fitness utilizado es:

$$F = 500 - 35d + 4p - 30c - 3r + 2000m$$

Donde d es la distancia Manhattan a la meta, p los pasos utiles, c los choques o movimientos bloqueados, r las posiciones repetidas y m vale 1 cuando se alcanza la meta y 0 en otro caso.

En cada generacion:

1. Se evalua toda la poblacion.
2. Se ordenan los individuos por fitness.
3. Se conservan seis elites.
4. Se seleccionan padres entre los mejores individuos.
5. Se aplica cruce en un punto y mutacion gaussiana sobre los pesos.
6. Se completa la siguiente poblacion hasta llegar a 40 individuos.

Por defecto se ejecutan 40 generaciones y la semilla es aleatoria. Se puede fijar una semilla para reproducir un experimento concreto.

6. Ejecutar el laboratorio

Ejecucion normal, con 40 generaciones, semilla aleatoria y animacion:

```
python3 robot_red_neuronal_evolutiva.py
```

Ejecucion reproducible y sin pausa entre cuadros:

```
python3 robot_red_neuronal_evolutiva.py \\  
--semilla 7 --generaciones 40 --sin-pausa
```

La animacion del mejor individuo aparece despues de la evolucion. Cada cuadro muestra la observacion binaria, la accion elegida y la posicion actual.

7. Guardar y cargar el mejor individuo

Para entrenar y guardar la red:

```
python3 robot_red_neuronal_evolutiva.py \\  
--generaciones 40 --guardar-mejor mejor_red.pt
```

El archivo `mejor_red.pt` contiene los pesos de PyTorch y los metadatos basicos de la red. Para cargarlo sin evolucionar otra vez:

```
python3 robot_red_neuronal_evolutiva.py \\  
--cargar-red mejor_red.pt
```

Guardar la red separa dos fases del trabajo: primero se aprende una politica y despues se prueba esa misma politica bajo nuevas condiciones.

8. Probar en un mapa distinto

1. Ejecuta una evolucion y guarda `mejor_red.pt`.
2. Cambia manualmente `OBSTACULOS`, `INICIO` o `META` en el archivo.
3. Conserva el mismo `TAMANO` y evita colocar inicio o meta sobre un obstaculo.
4. Ejecuta con `-cargar-red mejor_red.pt`.
5. Registra si la red llega, cuantos pasos usa, cuantos choques produce y si entra en un ciclo.

Para que la comparacion sea valida, no se deben cambiar los pesos entre las dos pruebas. Solo se modifica el mapa.

1. Que se conserva entre la clase de percepcion local y este laboratorio?
2. Que informacion representa un peso de la red que antes representaba una regla?
3. Por que una red entrenada en un mapa puede fallar en otro mapa?
4. La politica aprendida generaliza o solo memoriza respuestas locales?
5. Que cambios permitirian que el robot conociera la direccion de la meta?

9. Experimento y tabla de resultados

Realiza tres entrenamientos con semillas distintas. Guarda cada mejor individuo con un nombre diferente y prueba cada uno en el mapa original y en un mapa modificado.

Semilla	Mapa	Llego	Generacion	Fitness	Choques	Pasos utiles
7	Original					
7	Modificado					
21	Original					
21	Modificado					
42	Original					
42	Modificado					

10. Entrega

Entrega los siguientes elementos:

- `robot_red_neuronal_evolutiva.py`.
- Un archivo `.pt` con al menos un mejor individuo.
- La tabla de resultados completa.
- Una captura o descripcion de la trayectoria en el mapa original y en el modificado.
- Una conclusion de 8 a 10 lineas sobre evolucion, percepcion local y generalizacion.

11.Codigo completo

```
import argparse
import os
import random
import time
from copy import deepcopy

import torch
import torch.nn as nn

# =====
# 1. CONFIGURACION DEL PROBLEMA
# =====
# Aca definimos el universo del robot: el mapa de 8x8, el punto de inicio,
# la meta y los obstaculos. Toda la dinamica del problema queda encapsulada
# aqui para que la red y el algoritmo genetico trabajen sobre el mismo entorno
# que en las clases anteriores.
#
# La clave pedagogica es que la representacion del individuo cambia: ya no es
# una lista de acciones como en la version reactiva, sino una red neuronal que
# recibe la observacion local y devuelve la accion a tomar.
COMANDOS = ("U", "D", "L", "R")
ORDEN_SENSORES = ("U", "D", "L", "R")

TAMANO = 8
INICIO = (0, 0)
META = (7, 7)

OBSTACULOS = {
    (0, 3), (1, 3), (2, 0), (2, 2), (2, 3),
    (4, 5), (5, 2), (5, 3), (5, 4),
    (6, 6),
```

```

}

# La cantidad de pasos y el tamaño de la población están pensados para que el
# experimento avance rápido sin perder legibilidad. Son valores modestos pero
# suficientes para que el algoritmo evolutivo empiece a funcionar bien.
PASOS_MAXIMOS = 30
TAMANO_POBLACION = 40
ELITE = 6
PADRES = 16

EMOJI_ROBOT = "\U0001f916"
EMOJI_META = "\U0001f7e9"
EMOJI_META_ALCANZADA = "\U0001f389"
EMOJI Obstaculo = "\u2b1b"
EMOJI_INICIO = "\U0001f535"
EMOJI_LIBRE = "\u2b1c"

# =====
# 2. MODELO NEURONAL (PYTORCH)
# =====
# La idea del laboratorio es que la política no sea una tabla 16x1, sino una
# red capaz de generalizar aun cuando la situación cambia. El robot observa 4
# sensores localmente: si hay obstaculo hacia arriba, abajo, izquierda o
# derecha. Esa observación se transforma en un vector de 4 elementos que entra a
# la red. La red produce 4 salidas, una por cada acción posible: U, D, L y R.
#
# Elegimos la acción con mayor valor en la salida (argmax). Esto da una forma
# muy sencilla de emular la toma de decisiones del robot con una política
# parametrizada por pesos.
class RedPercepcion(nn.Module):
    """Pequeña red con 4 sensores de entrada y 4 acciones de salida."""

    def __init__(self, hidden_size=8):
        super().__init__()
        # Capa oculta: combina la información sensorial local y la transforma en
        # representaciones internas. 8 neuronas es suficiente para una versión
        # didáctica y muy ligera.
        self.hidden = nn.Linear(4, hidden_size)
        # Capa de salida: 4 logits, uno por acción posible.
        self.out = nn.Linear(hidden_size, 4)

    def forward(self, x):
        # ReLU introduce no-linealidad. Sin ella, la red sería un simple mapeo
        # lineal y perdería gran parte de la capacidad representativa.
        return self.out(torch.relu(self.hidden(x)))

    def vector(self):
        # Convierte todos los parámetros de la red en un único tensor 1D para
        # poder aplicarle mutaciones y cruce como si fueran genes.
        return torch.cat([p.detach().reshape(-1) for p in self.parameters()])

    def cargar_vector(self, vector):
        # Permite reconstruir una red a partir de un vector plano. Esto es
        # útil para dar un paso de evolución a peso, como si cada peso fuera un
        # gene de un individuo.
        with torch.no_grad():
            pos = 0
            for parametro in self.parameters():
                largo = parametro.numel()
                parametro.copy_(vector[pos:pos + largo].reshape_as(parametro))
                pos += largo

def crear_red_aleatoria(rng, hidden_size=8):

```

```

"""Inicializa un individuo con pesos aleatorios."""
red = RedPercepcion(hidden_size=hidden_size)
for parametro in red.parameters():
    # La distribucion gaussiana centrada en 0 ayuda a que la red no empiece
    # con pesos extremadamente grandes ni saturados.
    parametro.data.normal_(mean=0.0, std=0.5)
return red

def copiar_red(red):
    """Crea una copia profunda de la red para no mezclar estados."""
    copia = RedPercepcion(hidden_size=8)
    copia.load_state_dict(deepcopy(red.state_dict()))
    return copia

def guardar_red(red, ruta):
    """Guarda un individuo entrenado en disco para reutilizarlo despues."""
    directorio = os.path.dirname(ruta)
    if directorio:
        os.makedirs(directorio, exist_ok=True)

    estado = {
        "state_dict": red.state_dict(),
        "hidden_size": 8,
        "comandos": COMANDOS,
        "orden_sensores": ORDEN_SENSORES,
        "tamano": TAMANO,
        "inicio": INICIO,
        "meta": META,
        "obstaculos": sorted(OBSTACULOS),
    }
    torch.save(estado, ruta)
    return ruta

def cargar_red(ruta):
    """Carga una red previamente guardada desde disco."""
    checkpoint = torch.load(ruta, map_location="cpu")
    if isinstance(checkpoint, nn.Module):
        return checkpoint

    red = RedPercepcion(hidden_size=checkpoint.get("hidden_size", 8))
    red.load_state_dict(checkpoint["state_dict"])
    red.eval()
    return red

# =====
# 3. ENTORNO Y SENSADO
# =====
# Esta seccion es casi identica a la version reactiva: el robot sigue usando el
# mismo mapa, sigue teniendo los mismos obstaculos y la misma copia del mundo.
# Lo que cambia aqui es la forma de convertir la sensacion en accion.
#
# En lugar de mirar una regla fija del ADN, la red observa los cuatro sensores y
# decide. El entorno sigue imponiendo las reglas: si el movimiento sale del
# tablero o choca con un obstaculo, el robot no avanza.
def limpiar_pantalla():
    os.system("cls" if os.name == "nt" else "clear")

def destino_de(posicion, comando):
    """Devuelve la celda siguiente en la direccion indicada."""
    fila, columna = posicion

```

```

cambios = {"U": (-1, 0), "D": (1, 0), "L": (0, -1), "R": (0, 1)}
cambio_fila, cambio_columna = cambios[comando]
return fila + cambio_fila, columna + cambio_columna

def esta_bloqueada(posicion, obstaculos=None):
    """Verifica si una celda es invalida: fuera del tablero u obstaculo."""
    fila, columna = posicion
    obstaculos = OBSTACULOS if obstaculos is None else set(obstaculos)
    return not (0 <= fila < TAMANO and 0 <= columna < TAMANO) or posicion in obstaculos

def observar(posicion, obstaculos=None):
    """Codifica los cuatro sensores en un entero de 4 bits."""
    patron = 0
    for comando in ORDEN_SENSORES:
        patron <<= 1
        patron |= int(esta_bloqueada(destino_de(posicion, comando), obstaculos=obstaculos))
    return patron

def decidir(red, posicion, obstaculos=None):
    """Pasa la observacion por la red y devuelve la accion escogida."""
    # La observacion se convierte en una secuencia de bits [U, D, L, R] con la
    # forma de un vector de 4 entradas. Esto es lo que se le da a la red.
    bits = [(observar(posicion, obstaculos=obstaculos) >> (3 - i)) & 1 for i in range(4)]
    entrada = torch.tensor(bits, dtype=torch.float32).unsqueeze(0)
    with torch.no_grad():
        logits = red(entrada)[0]
    indice = int(torch.argmax(logits).item())
    return COMANDOS[indice]

def mover(posicion, comando, obstaculos=None, tamano=None):
    """Intenta mover al robot y devuelve si hubo avance, choque o borde."""
    destino = destino_de(posicion, comando)
    tamano = TAMANO if tamano is None else tamano
    obstaculos = OBSTACULOS if obstaculos is None else set(obstaculos)
    if not (0 <= destino[0] < tamano and 0 <= destino[1] < tamano):
        return posicion, "borde"
    if destino in obstaculos:
        return posicion, "obstaculo"
    return destino, "avance"

def recorrer(red, obstaculos=None, inicio=None, meta=None, pasos_maximos=None, tamano=None):
    """Ejecuta una trayectoria completa para un individuo concreto."""
    obstaculos = OBSTACULOS if obstaculos is None else set(obstaculos)
    inicio = INICIO if inicio is None else inicio
    meta = META if meta is None else meta
    pasos_maximos = PASOS_MAXIMOS if pasos_maximos is None else pasos_maximos
    tamano = TAMANO if tamano is None else tamano

    posicion = inicio
    trayectoria = [posicion]
    observaciones = []
    acciones = []
    choques = 0
    pasos utiles = 0

    for _ in range(pasos_maximos):
        observacion = observar(posicion, obstaculos=obstaculos)
        comando = decidir(red, posicion, obstaculos=obstaculos)
        nueva_posicion, resultado = mover(posicion, comando, obstaculos=obstaculos, tamano=tamano)

```

```

observaciones.append(observacion)
acciones.append(comando)

if resultado in ("borde", "obstaculo"):
    choques += 1
if resultado == "avance":
    pasos utiles += 1

posicion = nueva_posicion
trayectoria.append(posicion)
if posicion == meta:
    break

return trayectoria, choques, pasos utiles, observaciones, acciones

# =====
# 4. FITNESS Y EVOLUCION
# =====
# El algoritmo genetico necesita una medida que permita ordenar a los
# individuos: el fitness. En este caso, se aprovecha la misma idea que en los
# laboratorios previos para mantener continuidad didactica.
#
# Cuanto mejor sea la politica, mas puntuacion recibe. Se premia llegar a la
# meta, evitar choques, avanzar y no repetir posiciones innecesariamente.
def evaluar(red, obstaculos=None, inicio=None, meta=None, pasos_maximos=None, tamano=None):
    obstaculos = OBSTACULOS if obstaculos is None else set(obstaculos)
    inicio = INICIO if inicio is None else inicio
    meta = META if meta is None else meta
    pasos_maximos = PASOS_MAXIMOS if pasos_maximos is None else pasos_maximos
    tamano = TAMANO if tamano is None else tamano

    trayectoria, choques, pasos utiles, _, _ = recorrer(red, obstaculos=obstaculos, inicio=inicio,
    meta=meta, pasos_maximos=pasos_maximos, tamano=tamano)
    posicion = trayectoria[-1]
    distancia = abs(meta[0] - posicion[0]) + abs(meta[1] - posicion[1])
    visitas_repetidas = len(trayectoria) - len(set(trayectoria))

    puntaje = 500
    puntaje -= distancia * 35
    puntaje += pasos utiles * 4
    puntaje -= choques * 30
    puntaje -= visitas_repetidas * 3

    if posicion == meta:
        puntaje += 2000
    return puntaje

def mutar_red(red, tasa_mutacion, sigma, rng):
    """Aplica ruido gaussiano a algunos pesos, dejando el resto intacto."""
    # Cada individuo es una red, y cada peso de esa red puede mutar con cierta
    # probabilidad. La idea es similar a la mutacion de genes en un ADN, pero
    # ahora sobre los parametros numericos de la red.
    red_mutada = copiar_red(red)
    with torch.no_grad():
        for parametro in red_mutada.parameters():
            ruido = torch.randn_like(parametro) * sigma
            mascara = torch.rand_like(parametro) < tasa_mutacion
            parametro.add_(ruido * mascara.to(parametro.dtype))
    return red_mutada

def cruzar_redes(padre_1, padre_2, rng):
    """Combina dos redes a partir de una particion de su vector de pesos."""

```

```

# Convertimos todos los pesos a una secuencia plana y mezclamos una parte
# de cada padre. Esta tecnica es una version directa de cruza en un punto,
# pero aplicada a pesos de una red neuronal.
v1 = padre_1.vector()
v2 = padre_2.vector()
punto = rng.randint(1, len(v1) - 1)
hijo = v1.clone()
hijo[punto:] = v2[punto:]

red_hijo = RedPercepcion(hidden_size=8)
red_hijo.cargar_vector(hijo)
return red_hijo

def evolucionar(tasa_mutacion=0.15, usar_cruce=True, semilla=None, maximo_generaciones=40):
    """Ejecuta el ciclo evolutivo completo y devuelve el mejor individuo."""
    # Si la semilla es None, se genera una aleatoria. Esto hace que la ejecucion
    # sea mas realista y no siempre repita la misma secuencia para cada ejecucion.
    if semilla is None:
        semilla = random.randint(1, 1000000)
    rng = random.Random(semilla)
    poblacion = [crear_red_aleatoria(rng) for _ in range(TAMANO_POBLACION)]
    historial = []

    for generacion in range(maximo_generaciones + 1):
        # Ordenamos por fitness para que el mejor quede primero.
        poblacion.sort(key=evaluar, reverse=True)
        mejor = poblacion[0]
        puntaje = evaluar(mejor)
        trayectoria, choques, pasos utiles, _, _ = recorrer(mejor)
        historial.append((generacion, puntaje, len({str(r.vector().tolist()) for r in poblacion})))

        # Si la mejor red llega a la meta, cerramos la evolucion antes de gastar
        # generaciones extras.
        if trayectoria[-1] == META:
            return {
                "metodo": "red neuronal + evolutivo",
                "red": mejor,
                "generacion": generacion,
                "puntaje": puntaje,
                "choques": choques,
                "pasos utiles": pasos utiles,
                "llego": True,
                "historial": historial,
                "poblacion": poblacion,
            }

        # Elitismo: se preservan los mejores individuos, que son la base del
        # siguiente ciclo generacional.
        nueva_poblacion = [copiar_red(red) for red in poblacion[:ELITE]]
        while len(nueva_poblacion) < TAMANO_POBLACION:
            madre = rng.choice(poblacion[:PADRES])
            if usar_cruce and rng.random() < 0.75:
                padre = rng.choice(poblacion[:PADRES])
                hijo = cruzar_redes(madre, padre, rng)
            else:
                hijo = copiar_red(madre)
            hija = mutar_red(hijo, tasa_mutacion, sigma=0.45, rng=rng)
            nueva_poblacion.append(hija)
        poblacion = nueva_poblacion

    # Si llegamos al final del limite de generaciones, devolvemos el mejor
    # individuo disponible.
    poblacion.sort(key=evaluar, reverse=True)
    mejor = poblacion[0]

```

```

trayectoria, choques, pasos_utiles, _, _ = recorrer(mejor)
return {
    "metodo": "red neuronal + evolutivo",
    "red": mejor,
    "generacion": maximo_generaciones,
    "puntaje": evaluar(mejor),
    "choques": choques,
    "pasos_utiles": pasos_utiles,
    "llego": trayectoria[-1] == META,
    "historial": historial,
    "poblacion": poblacion,
}

# =====
# 5. VISUALIZACION
# =====
# Esta parte es exclusivamente didactica: dibuja el mapa, ubica al robot y
# anima cada paso de su recorrido. Es util para mostrar que la red no solo
# devuelve un resultado, sino que toma decisiones secuenciales en tiempo real.
def dibujar(red, pausa=0.1, titulo="MEJOR INDIVIDUO", obstaculos=None, inicio=None, meta=None,
    pasos_maximos=None, tamano=None):
    obstaculos = OBSTACULOS if obstaculos is None else set(obstaculos)
    inicio = INICIO if inicio is None else inicio
    meta = META if meta is None else meta
    pasos_maximos = PASOS_MAXIMOS if pasos_maximos is None else pasos_maximos
    tamano = TAMANO if tamano is None else tamano

    trayectoria, _, _, observaciones, acciones = recorrer(red, obstaculos=obstaculos, inicio=inicio
    , meta=meta, pasos_maximos=pasos_maximos, tamano=tamano)

    for paso, posicion in enumerate(trayectoria[1:], start=1):
        limpiar_pantalla()
        print(f"RED NEURONAL EVOLUTIVA | {titulo} | Puntaje: {evaluar(red, obstaculos=obstaculos,
        inicio=inicio, meta=meta, pasos_maximos=pasos_maximos, tamano=tamano)}")
        print(f"{EMOJI_ROBOT} = robot | {EMOJI_META_ALCANZADA} = meta alcanzada | {EMOJI_META} =
        meta | {EMOJI_OBSTACULO} = obstaculo")
        print(f"{EMOJI_INICIO} = inicio | {EMOJI_LIBRE} = celda libre\n")

        for fila in range(tamano):
            linea = ""
            for columna in range(tamano):
                celda = (fila, columna)
                if celda == posicion:
                    linea += EMOJI_ROBOT
                elif celda == inicio:
                    linea += EMOJI_INICIO
                elif celda == meta:
                    linea += EMOJI_META_ALCANZADA if posicion == meta else EMOJI_META
                elif celda in obstaculos:
                    linea += EMOJI_OBSTACULO
                else:
                    linea += EMOJI_LIBRE
            print(linea)

        print(f"Paso {paso}/{len(trayectoria)-1} | Observacion: {observaciones[paso - 1]:04b} |
        Accion: {acciones[paso - 1]} | Mejor individuo")
        if pausa:
            time.sleep(pausa)

def main():
    """Punto de entrada del laboratorio: lee parametros y ejecuta la simulacion."""
    parser = argparse.ArgumentParser(description="Red neuronal pequena evolucionada por algoritmo
    genetico.")

```

```

parser.add_argument("--semilla", type=int, default=None, help="Semilla del generador aleatorio. Si no se pasa, se elige aleatoria.")
parser.add_argument("--generaciones", type=int, default=40, help="Cuántas generaciones evolutivas se ejecutan por defecto.")
parser.add_argument("--tasa-mutacion", type=float, default=0.15, help="Probabilidad de mutación de cada peso.")
parser.add_argument("--sin-pausa", action="store_true", help="Si se activa, la animación se ejecuta sin esperar entre pasos.")
parser.add_argument("--guardar-mejor", type=str, default=None, help="Ruta donde guardar el mejor individuo encontrado.")
parser.add_argument("--cargar-red", type=str, default=None, help="Ruta de una red previa para probarla sin volver a evolucionar.")
args = parser.parse_args()

pausa = 0 if args.sin_pausa else 0.08

if args.cargar_red:
    red = cargar_red(args.cargar_red)
    resultado = {
        "metodo": "red neuronal cargada",
        "red": red,
        "generacion": "cargado",
        "puntaje": evaluar(red),
        "choques": recorrer(red)[1],
        "pasos utiles": recorrer(red)[2],
        "llego": recorrer(red)[0][-1] == META,
    }
else:
    resultado = evolucionar(
        tasa_mutacion=args.tasa_mutacion,
        usar_cruce=True,
        semilla=args.semilla,
        maximo_generaciones=args.generaciones,
    )

    if args.guardar_mejor:
        guardar_red(resultado["red"], args.guardar_mejor)
        print(f"\nMejor individuo guardado en: {args.guardar_mejor}")

if args.cargar_red and args.guardar_mejor:
    guardar_red(resultado["red"], args.guardar_mejor)
    print(f"\nRed cargada guardada de nuevo en: {args.guardar_mejor}")

print("\nANIMACION DEL MEJOR INDIVIDUO")
dibujar(resultado["red"], pausa=pausa, titulo="MEJOR INDIVIDUO")
print("\nRESULTADO FINAL")
print(f"Metodo: {resultado['metodo']}")
print(f"Generacion: {resultado['generacion']}")
print(f"Puntaje: {resultado['puntaje']}")
print(f"Choques: {resultado['choques']}")
print(f"Pasos utiles: {resultado['pasos utiles']}")
print(f"Llegó a la meta: {'si' if resultado['llego'] else 'no'}")

if __name__ == "__main__":
    main()

```